Experiment 6

Aim

To implement and analyze the **Support Vector Machine (SVM)** classification model on the **Pima Indians Diabetes Dataset** using Python and improve model performance using **hyperparameter tuning**.

---

1. Dataset Source

Pima Indians Diabetes Dataset

Target Variable: Outcome

Source:
https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

---

2. Dataset Description

The **Pima Indians Diabetes Dataset** contains medical diagnostic measurements used to predict whether a patient has diabetes. It contains **768 records and 9 attributes**, where the target variable indicates diabetes presence (0 or 1).

Features

- Pregnancies — Number of times pregnant
- Glucose — Plasma glucose concentration (mg/dL)
- BloodPressure — Diastolic blood pressure (mm Hg)
- SkinThickness — Triceps skin fold thickness (mm)
- Insulin — 2-Hour serum insulin (mu U/ml)
- BMI — Body Mass Index (kg/m²)
- DiabetesPedigreeFunction — Genetic diabetes likelihood

- Age — Age of patient

Target

- 0 → Non-diabetic
- 1 → Diabetic

This dataset is commonly used for binary classification problems in healthcare prediction models.

---

3. Mathematical Formulation of the Algorithm

SVM is a **supervised machine learning algorithm** used for classification tasks. It finds the **optimal hyperplane** that separates different classes with maximum margin.

---

1. Hyperplane Equation

$$w \cdot x + b = 0$$

Where:

w = weight vector
x = feature vector
b = bias term

---

2. Margin Maximization

The distance between the hyperplane and nearest data points is maximized.

$$Margin = \frac{2}{||w||}$$

The closest points to the hyperplane are called **Support Vectors**.

---

3. Kernel Functions

SVM can handle **non-linear data** using kernel transformations.

Common kernels:

- Linear Kernel
- Polynomial Kernel
- Radial Basis Function (RBF)
- Sigmoid Kernel

---

4. Algorithm Limitations

- Selecting the proper **kernel and parameters** can be difficult.
- Computationally expensive for large datasets.
- Requires **feature scaling** for best performance.
- Model interpretation is harder than simple algorithms.

---

5. Methodology / Workflow

Common Workflow

1. Load dataset using **pandas**
2. Separate **independent variables (X)** and **target variable (y)**
3. Apply **feature scaling** using StandardScaler
4. Split dataset into **training and testing sets**
5. Train **SVM classifier**
6. Perform **hyperparameter tuning using GridSearchCV**
7. Predict outcomes on test data
8. Evaluate model performance using classification metrics

---

6. Performance Analysis

1. **Accuracy Score**
   Measures overall correctness of the model.
2. **Confusion Matrix**
   Shows correct and incorrect predictions.
3. **Precision**
   Measures proportion of correctly predicted positive cases.
4. **Recall**
   Measures the model's ability to detect actual positives.
5. **F1-Score**
   Balances precision and recall.

---

7. Hyperparameter Tuning

Parameter	Meaning
C	Regularization parameter controlling margin
kernel	Type of kernel function
gamma	Influence of training samples
degree	Degree of polynomial kernel

Exercise 1: Support Vector Machine Classification with Hyperparameter Tuning

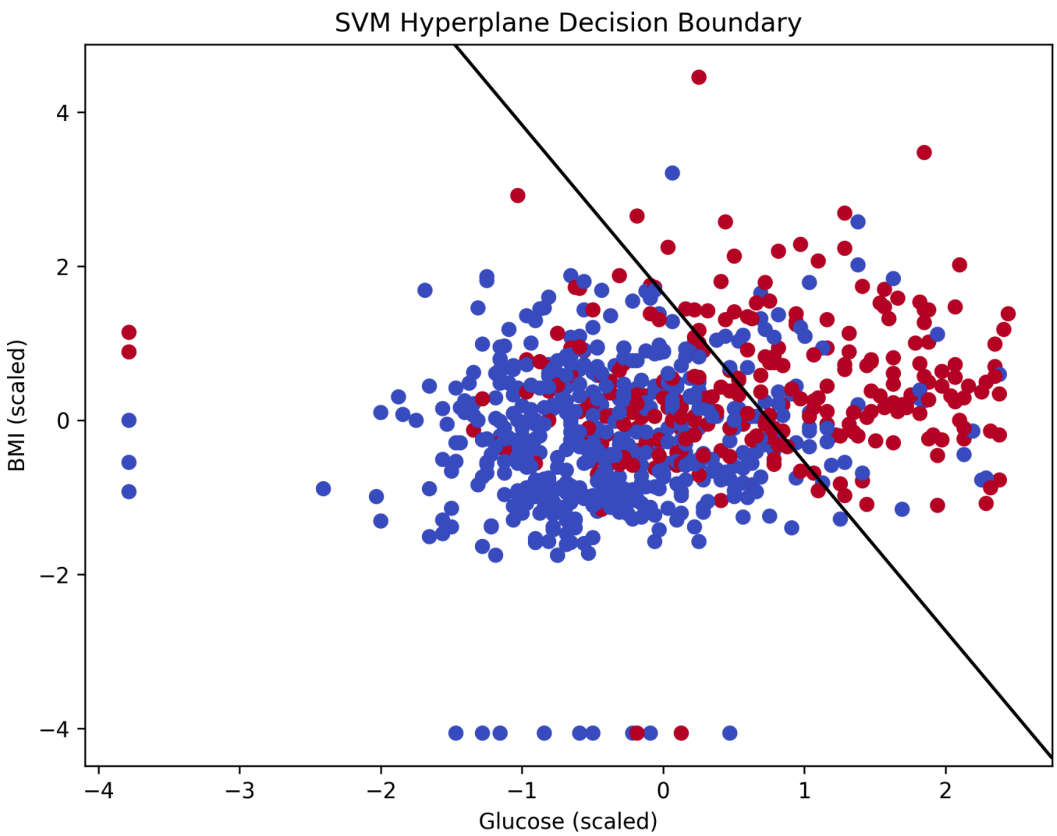
Code

```
import pandas as pd
from sklearn.model_selection import train_test_split,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, confusion_matrix,
classification_report

# Load dataset
df = pd.read_csv("diabetes.csv")   # Kaggle dataset

# Feature and target selection
X = df.drop("Outcome", axis=1)
y = df["Outcome"]
# Feature scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)
# Define SVM model
svm_model = SVC()
# Hyperparameter grid
param_grid = {
    "C": [0.1, 1, 10, 100],
    "kernel": ["linear", "rbf", "poly"],
    "gamma": ["scale", "auto"]
}
grid = GridSearchCV(svm_model, param_grid, cv=5)
grid.fit(X_train, y_train)
best_model = grid.best_estimator_

y_pred = best_model.predict(X_test)
print("Best Parameters:", grid.best_params_)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test,
y_pred))
```

Output

```
Best Parameters: {'C': 1, 'gamma': 'scale', 'kernel': 'rbf'}
Accuracy: 0.7272727272727273
Confusion Matrix:
 [[81 18]
 [24 31]]
Classification Report:
              precision    recall  f1-score   support

           0       0.77      0.82      0.79        99
           1       0.63      0.56      0.60        55

    accuracy                           0.73       154
   macro avg       0.70      0.69      0.70       154
weighted avg       0.72      0.73      0.72       154
```



Conclusion

This experiment implemented the **Support Vector Machine (SVM)** classification model using the **Pima Indians Diabetes Dataset**. The model was trained to predict whether a patient has diabetes based on medical diagnostic measurements. Feature scaling improved model performance, and **GridSearchCV** was used to tune hyperparameters such as **C, kernel, and gamma**. Evaluation metrics like **accuracy, confusion matrix, precision, recall, and F1-score** helped assess the model's performance in predicting diabetes.